DIO Driver User Guide

Traveo™ II Family

About this document

Scope and purpose

This user guide describes the architecture, configuration, and use of the Digital input/output (DIO) Driver. This guide also explains the functionality of the driver and provides a reference for the driver's API.

The installation, build process, and general information about the use of the EB tresos Studio are not within the scope of this document. Refer to the *EB tresos Studio for ACG8 User's Guide* **[7]** for a detailed description of these topics.

Intended audience

This document is intended for anyone who uses the digital input/output (DIO) driver of the Traveo II family.

Document Structure

Chapter **1 General Overview** gives a brief introduction to the DIO Driver, explains the embedding of the driver in the AUTOSAR environment and describes the supported hardware and development environment.

Chapter **2 Using the DIO Driver** details the steps required to use the DIO Driver in your application.

Chapter **3 Structure and Dependencies** describes the file structure and the dependencies for the DIO Driver.

Chapter **4 EB tresos Studio Configuration Interface** describes the driver's configuration with the EB tresos Studio.

Chapter **5 Functional Description** gives a functional description of all services offered by the DIO Driver.

Chapter **6 Hardware Resources** describes the hardware resources used.

The Appendix provides a complete API reference and Access Register Table.

Abbreviations and definitions

Table 1 Abbreviations

Words and Terms	Description
API	Application Programming Interface
ASIL	Automotive Safety Integrity Level
AUTOSAR	Automotive Open System Architecture
BSW	Basic Software. Standardized part of software which does not fulfill a vehicle functional job.
Channel	Represents a single general purpose digital input/output pin.
Channel Group	A set of adjoining channels, all within a given port, which can be accessed as a single entity by its group name.
DEM	Diagnostic Event Manager
DET	Default Error Tracer

About this document

Words and Terms	Description
DIO	Digital input/output
EB tresos Studio	Elektrobit Automotive configuration framework
HW	Hardware
LSB	Least Significant Bit
MCAL	Microcontroller Abstraction Layer
MCU	Microcontroller Unit
MSB	Most Significant Bit
Port	Represents several DIO Channels that are grouped by hardware, typically controlled by one hardware register.
SW	Software
UTF-8	8-Bit Universal Character Set Transformation Format
µC	Microcontroller

Related Documents

AUTOSAR Requirements and Specifications

Bibliography

[1] General Specification of Basic Software Modules, AUTOSAR Release 4.2.2.

[2] Specification of DIO Driver, AUTOSAR Release 4.2.2.

[3] Specification of PORT Driver, AUTOSAR Release 4.2.2.

[4] Specification of Standard Types, AUTOSAR Release 4.2.2.

[5] Specification of ECU Configuration Parameters, AUTOSAR Release 4.2.2.

[6] Specification of Default Error Tracer, AUTOSAR Release 4.2.2.

Elektrobit Automotive Documentation

Bibliography

[7] EB tresos Studio for ACG8 User's Guide.

Hardware Documentation

The hardware documents are listed in the delivery notes.

Related Standards and Norms

Bibliography

[8] Layered Software Architecture, AUTOSAR Release 4.2.2.

Table of contents

Table of contents

1 General Overview

1.1 Introduction to the DIO Driver

The DIO Driver is a set of software routines, which enables the reading and writing of the µC hardware's general-purpose input and output pins.

For this purpose, it provides services for modifying the levels of the following:

- DIO Channels (Pins)
- DIO Ports
- DIO Channel Groups

The DIO Driver is not responsible for initializing or configuring the mode (DIO, PWM, ADC, ...) of the hardware ports. This is the task of the PORT Driver. The DIO Driver does not provide the `DIO_Init()` function.

The driver conforms to the AUTOSAR standard and is implemented according to the *Specification of DIO Driver* [2].

Furthermore, the DIO Driver is delivered with a plugin for the EB tresos Studio, which can be used for static configuration of the driver. The driver also provides an interface to enable ports and channels, to define symbolic names, and to create channel groups.

1.2 User Profile

This guide is intended for users with a basic knowledge of the following:

- Embedded systems
- The C programming language
- The AUTOSAR standard
- The target hardware architecture

1.3 Embedding in the AUTOSAR Environment

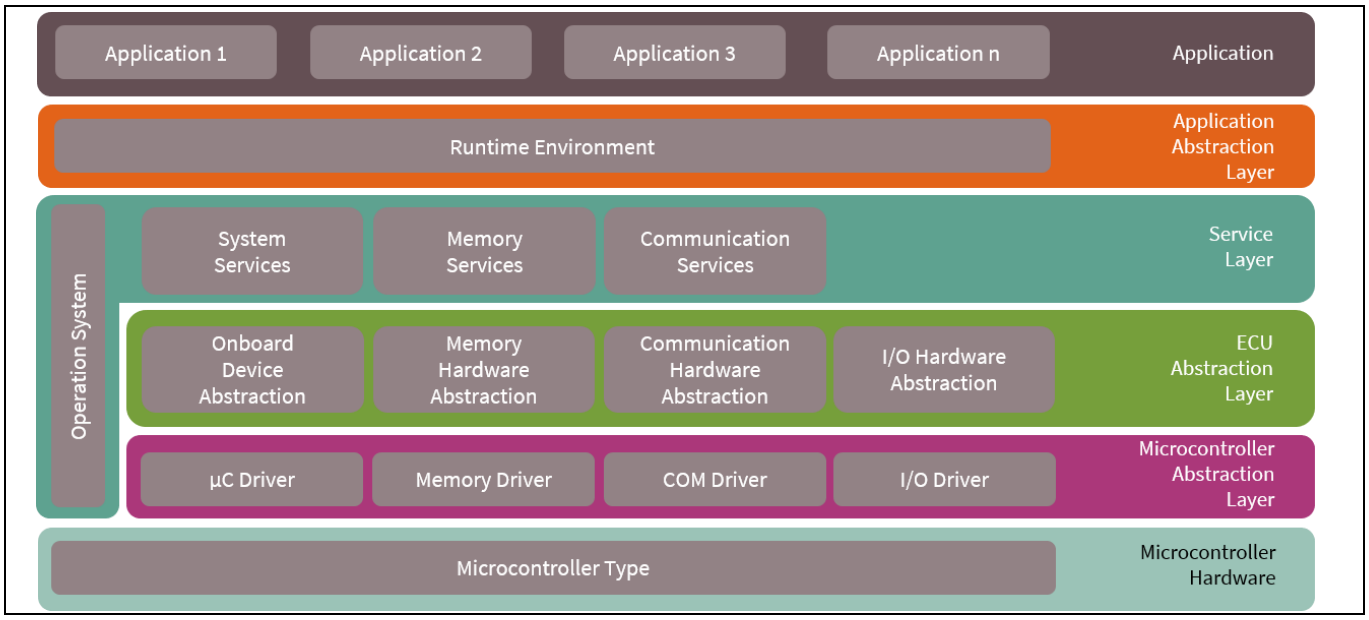

Figure 1 Overview of AUTOSAR Software Layers

Figure 1 depicts the layered AUTOSAR software architecture. The DIO Driver (**Figure 2**) is part of the Microcontroller Abstraction Layer (MCAL), the lowest layer of Basic Software in the AUTOSAR environment.

As an internal I/O driver, it provides a standardized and µC independent interface to higher software layers for accessing digital input/output pins of the ECU hardware.

For a thorough overview of the AUTOSAR layered software architecture, refer to *Layered Software Architecture* [8].

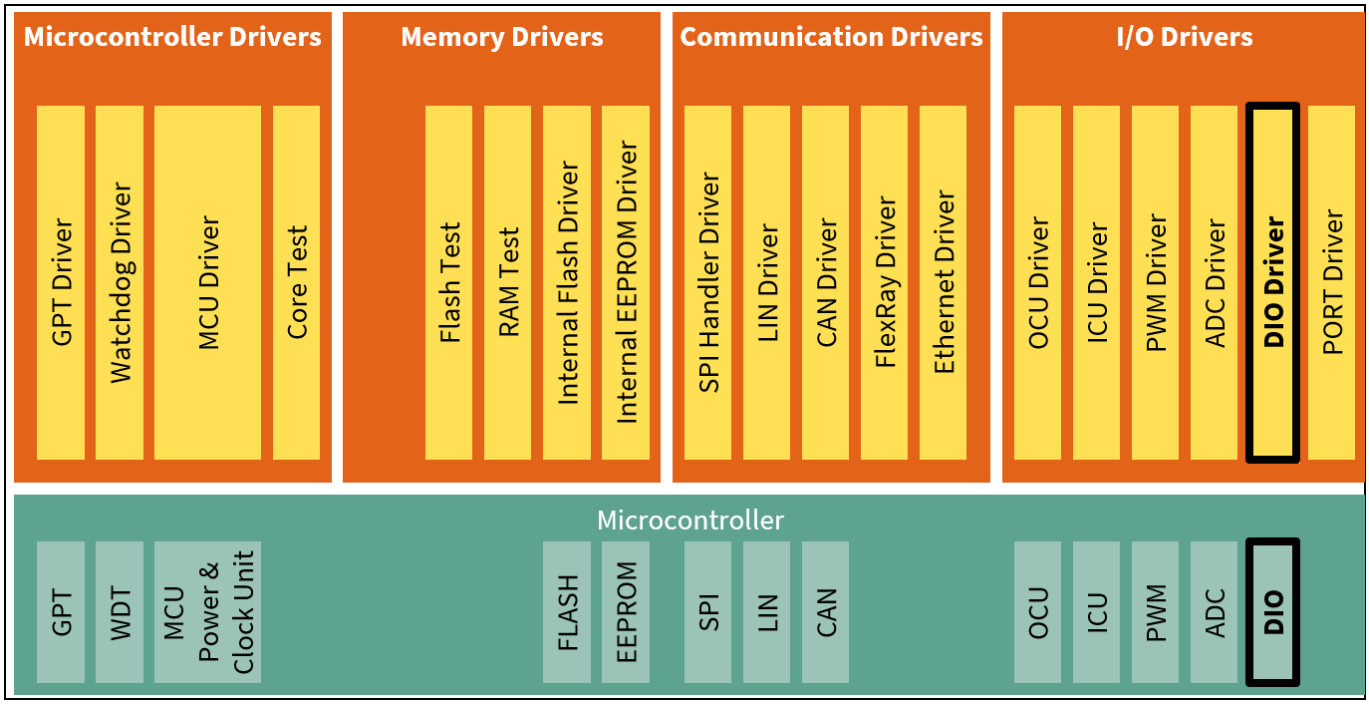

Figure 2 DIO Driver in MCAL Layer

1.4 Supported Hardware

This version of the DIO Driver supports the Traveo II microcontroller. Smaller derivatives have only a subset of the ports and pins defined for the microcontroller. The supported derivatives are listed in the release notes.

New derivatives will be supported on request or via resource file update. For an overview of all supported derivatives, refer to the Resource plugin. All additional derivatives not mentioned in this user guide are similar to or compatible with the base implementation of this driver.

1.5 Development Environment

The development environment corresponds to AUTOSAR release 4.2.2. The modules BASE, Make, PORT, and Resource are needed for proper functionality of the DIO Driver.

1.6 Character Set and Encoding

All source code files of the DIO Driver are restricted to the ASCII character set. The files are encoded in UTF-8 format, with only the 7-bit subset (values 0x00 … 0x7F) being used.

2 Using the DIO Driver

This chapter describes all the steps necessary to incorporate the DIO Driver into your application.

2.1 Installation and Prerequisites

Note: *Before you start, see the EB tresos Studio for ACG8 User's Guide [7] for the following information.*

1. *The installation procedure of EB tresos ECU AUTOSAR components.*
2. *The usage of the EB tresos Studio.*
3. *The usage of the EB tresos ECU AUTOSAR Build Environment (It includes an explanation of how to set up and integrate your application within the EB tresos ECU AUTOSAR Build Environment).*

The installation of the DIO Driver complies with the general installation procedure for EB tresos ECU AUTOSAR components given in the documents mentioned above. If the driver is successfully installed, it will appear in the module list of the EB tresos Studio (see *EB tresos Studio for ACG8 User's Guide* **[7]**).

In the following sections, it is assumed, that the project is properly set up and uses the application template as described in the *EB tresos Studio for ACG8 User's Guide* **[7]**. This template provides the necessary folder structure, project, and makefiles needed to configure and compile your application within the build environment. You need to be familiar with the usage of the command shell.

All needed port pins must be configured in the PORT Driver to use digital input/output functionality of the DIO Driver. Check the *Specification of PORT Driver* **[3]** for PORT Driver configuration.

2.2 Configuring the DIO Driver

This section gives a brief overview of the configuration structure defined by AUTOSAR to use the DIO Driver.

The following basic containers are used to configure common behavior:

- *DioGeneral*: This container is mainly used to restrict/extend the API of the DIO Driver and used to enable/disable DET.
- *DioConfig*: This container has the configuration parameters and sub containers of the AUTOSAR DIO Driver. It includes the *DioPort* containers.

For detailed information and description, see Chapter **4 EB tresos Studio Configuration Interface**.

The *DioConfig* container contains the *DioPort* containers, which define the ports including channels and channel groups for each port. Each *DioPort* has the `DioPortId` parameter, which assigns the configured port to the underlying hardware. Refer to each Hardware Manual for a list of hardware supported ports (refer to **Hardware Documentation** for the Hardware Manual version and subderivative).

For a *DioPort* , many *DioChannels* (single pins) can be configured by setting the `DioChannelId`. This value specifies the absolute position of the channel (starting with 0). A DIO Pin can be accessed later via its symbolic name and the read/write channel functions of the DIO Driver. The number of available pins in the port is described in each Hardware Manual (refer to **Hardware Documentation** for the Hardware Manual version and subderivative).

Note: *At first, configure the `DioChannelPin` to get the correct port pin number easily. Then press the calculate button of `DioChannelId` parameter to get the corresponding `DioChannelId`.*

A `DioChannelGroup` is defined by setting a bit mask in `DioPortMask`. All "1" bits in this consecutive mask define port pins considered in this channel group. The corresponding offset will be calculated automatically

during the generation process from the mask value. The number of channel groups within a *DioPort* is not restricted; overlapping groups are allowed. The group belongs to the parent *DioPort* container.

2.2.1 Architecture Specifics

See section **4 EB tresos Studio Configuration Interface**, where all configuration parameters are described.

2.3 Adapting your Application

To use the DIO Driver in your application, you must first include the DIO and the PORT Driver header files by adding the following code lines to your source file:

```
#include "Port.h"   /* PORT Driver */
#include "Dio.h"    /* DIO Driver  */
```

This publishes all needed types, function prototypes, and symbolic names of the configuration into the application.

You must also implement the error callout function for ASIL safety extension. Declare the error callout function in a specified file by the `DioIncludeFile` parameter and implement it in your application (see **7.1.5 Required Callback Functions**, Error callout API).

The `DioErrorCalloutFunction` parameter can configure the error callout function name.

The next step is to initialize and configure the ports. For this example, the DIO Port shall be configured as output. Otherwise the DIO Driver will exhibit undefined behavior. The PORT Driver User Guide explains how to configure the port with the PORT Driver in the EB tresos Studio.

The port initialization can be done with:

```
Port_Init(&Port_Config[0]);
```

After that, you can call API functions with the symbolic names defined in your configuration (such as LED_RED_PORT for the DioPort and LED_RED_CHANNEL_2 for bit 2 in this port).

Code Listing 1 Example LED Access

```
Dio_WritePort(DioConf_DioPort_LED_RED_PORT, 0xAA);
Dio_LevelType result = Dio_ReadChannel(DioConf_DioChannel_LED_RED_CHANNEL_2);
```

Available API functions are:

- `Dio_ReadChannel()` and `Dio_WriteChannel()` for channel (single pin) access.
- `Dio_ReadChannelGroup()` and `Dio_WriteChannelGroup()` for channel group access of consecutive pins in the port.
- `Dio_ReadPort()` and `Dio_WritePort()` for reading and writing the whole port.
- `Dio_FlipChannel()` for changing from 1 to 0 or from 0 to 1 the level of a channel.
- `Dio_MaskedWritePort()` for writing a subset of masked bits to a specified level.

More information about the API functions and data types is provided in **7 Appendix** and **5.4 Write Services**.

2.4 Starting the Build Process

Do the following to build your application:

Note: *For a clean build, use the build command with target `clean_all`. before (`make clean_all`).*

1. On the command shell, type the following command to generate the necessary configuration-dependent files. See **3.3 Generated Files**:

   ```
   > make generate
   ```

2. Type the following command to resolve the required file dependencies:

   ```
   > make depend
   ```

3. Type the following command to compile and link the application:

   ```
   > make (optional target: all)
   ```

The application is now built. All files are compiled and linked to a binary file, which can be downloaded to the target hardware.

2.5 Measuring Stack Consumption

Do the following to measure stack consumption. It requires the Base module for proper measurement.

Note: *All files (including library files) should be rebuilt with dedicated compiler option. The executable file built in this step must be used only to measure stack consumption.*

1. Add the following compiler option to the Makefile to enable stack consumption measurement.

   ```
   -DSTACK_ANALYSIS_ENABLE
   ```

2. Type the following command to clean library files.

   ```
   > make clean_lib
   ```

Follow the build process described in **2.4 Starting the Build Process**.

Follow the instructions in the release notes and measure the stack consumption.

2.6 Memory Mapping

Dio_MemMap.h in the *$(TRESOS_BASE)/plugins/MemMap_TS_T40D13M0I0R0/include* directory is a sample. This sample file is replaced by the file generated by the MEMMAP module. Input to the MEMMAP module is generated as *Dio_Bswmd.arxml* in the *$(PROJECT_ROOT)/ output/generated/swcd* directory of your project folder.

2.6.1 Memory Allocation Keyword

- DIO_START_SEC_CODE_ASIL_B / DIO_STOP_SEC_CODE_ASIL_B

  The memory section type is CODE. All executable code is allocated in this section.

- DIO_START_SEC_CONST_ASIL_B_UNSPECIFIED / DIO_STOP_SEC_CONST_ASIL_B_UNSPECIFIED

  The memory section type is CONST. The following constants are allocated in this section:

  - Port Configuration data
  - Hardware register base address data

3 Structure and Dependencies

The DIO Driver consists of static, configuration, and generated files.

3.1 Static Files

$(PLUGIN_PATH)=$(TRESOS_BASE)/plugins/Dio_TS_* is the path to the DIO Driver plugin.

$(PLUGIN_PATH)/lib_src contains all static source files of the DIO Driver. These files contain the functionality of the driver, which does not depend on the current configuration. The files are grouped into a static library.

$(PLUGIN_PATH)/lib_include contains all the internal header files for the DIO Driver.

$(PLUGIN_PATH)/src comprises configuration dependent source files or special derivative files. Each file will be built again when the configuration is changed.

All necessary source files will be automatically compiled and linked during the build process and all include paths will be set if the DIO Driver is enabled.

$(PLUGIN_PATH)/include is the basic public include directory that you need to include *Dio.h*.

$(PLUGIN_PATH)/autosar directory contains the AUTOSAR ECU parameter definition with vendor, architecture, and derivative specific adaptations to create a correct matching parameter configuration for the DIO Driver.

3.2 Configuration Files

The DIO Driver can be configured with EB tresos Studio. When saving a project, the configuration of the DIO Driver is saved in the file *Dio.xdm* in the *$(PROJECT_ROOT)/config* directory of your project folder. This file serves as an input for the generation of the configuration-dependent source and header files during the build process.

3.3 Generated Files

During the build process, the following files are generated based on the current configuration description. These files are in the sub folder *output/generated* of your project folder.

- *include/Dio_Cfg.h* provide all symbolic names of the configuration and are included by *Dio.h*.
- *include/Dio_Cfg_Internal.h* is a driver internal generated file including derivative-specific overall settings. This file is automatically included by the driver but not published via *Dio.h* because the settings are not public.

Note: *You do not need to add the generated source files to your application make file. They will be compiled and linked automatically during the build process.*

- *swcd/Dio_Bswmd.arxml* contains BswModuleDescription.

Note: *Additional steps are required for the generation of BSW module description.*
*In EB tresos Studio, follow the menu path **Project** > **Build Project** and select **generate_swcd**.*

3.4 Dependencies

3.4.1 DET

If the default error detection is enabled in the DIO Driver configuration, DET needs to be installed, configured, and built into the application.

This driver reports DET error codes as instance 0.

3.4.2 PORT Driver

Although the DIO Driver can be successfully compiled and linked without an AUTOSAR compliant PORT Driver, the latter is required to configure and to initialize all ports. Otherwise, the DIO Driver will show undefined behavior. Section **2.3 Adapting your Application** explains how you can add the PORT Driver to your application.

3.4.3 BSW Scheduler

The DIO Driver uses the following services of the BSW Scheduler to enter and leave critical sections:

- SchM_Enter_Dio_DIO_EXCLUSIVE_AREA_0(void)
- SchM_Exit_Dio_DIO_EXCLUSIVE_AREA_0(void)

Make sure that the BSW Scheduler is properly configured and initialized before using the DIO Driver.

3.4.4 Error Callout Handler

The error callout handler is called on every error that is detected regardless of whether the default error detection is enabled or disabled. The error callout handler is an ASIL safety extension that is not specified by AUTOSAR. It is configured via the configuration parameter `DioErrorCalloutFunction`.

4 EB tresos Studio Configuration Interface

The GUI is not part of the current delivery. For further information, see the *EB tresos Studio for ACG8 User's Guide* [7].

4.1 General Configuration

The module comes preconfigured with the default settings. These settings must be adapted when necessary.

- `DioDevErrorDetect` enables or disables the default error notification for the DIO Driver.

  Setting this parameter to false will disable the notification of development errors via DET. However, in contrast to AUTOSAR specification, detection of development errors is still enabled as safety mechanisms (fault detection).

- `DioVersionInfoApi` adds or removes the service `Dio_GetVersionInfo()` to/from the code.
- `DioFlipChannelApi` adds or removes the service `Dio_FlipChannel()` to/from the code.
- `DioMaskedWritePortApi` adds or removes the service `Dio_MaskedWritePort()` to/from the code.
- `DioErrorCalloutFunction` is used to specify the error callout function name. The function is called on every error. The ASIL level of this function limits the ASIL level of the DIO Driver.

Note: *`DioErrorCalloutFunction` must be a valid C function name, otherwise an error occurs in the configuration phase.*

- `DioIncludeFile` lists the file names that shall be included within the driver. Any application-specific symbol (such as the error callout function) that is used by the Dio configuration should be included by configuring this parameter.

Note: *`DioIncludeFile` must be a file name with a .h extension and a unique name, otherwise some errors occur during configuration.*

4.2 Port Configuration

- `DioPortId` is the port number.

  For example, the appropriate `DioPortId` for P000_1 or P009_3 are 0 and 9.

Note: *Since not all MCU ports may be used for DIO, there may be "gaps" in the list of all IDs.*

This value will be assigned to the following symbolic names:

- The symbolic name derived from the *DioPort* container short name prefixed with "*DioConf_DioPort_*".

4.3 Channel Configuration

4.3.1 Individual DIO Channel Configuration

Besides a HW-specific channel name, which is typically fixed for a specific micro controller, additional symbolic names can be defined per channel.

Note: *This container definition does not explicitly define a symbolic name parameter. Instead, the container's short name will be used in the ECU Configuration Description to specify the symbolic name of the channel.*

- `DioChannelId` is the port pin number. This value specifies the absolute position of the channel.

  For example, if you want to specify a pin number 0 of port number 1, set the 8 to `DioChannelId`. This means that `DioChannelId` is calculated by the following formula:

  `DioChannelId` of Pxxx_y = (xxx * 8) + y

  This value will be assigned to the following symbolic names.

  - The symbolic name derived from the *DioChannel* container short name prefixed with "*DioConf_DioChannel_*" .
- `DioChannelPin` is the port pin name.

  For example, the appropriate `DioChannelPin` for P000_1 or P022_7 are P000_1 and P022_7.

4.4 Channel Group Configuration

A channel group represents several adjoining DIO Channels represented by a logical group.

Note: *This container definition does not explicitly define a symbolic name parameter. Instead, the container's short name will be used in the ECU Configuration Description to specify the symbolic name of the channel group.*

- `DioChannelGroupIdentification` is identified in DIO API by a pointer to a data structure (of type `Dio_ChannelGroupType`). That data structure contains the channel group information. This parameter contains the code fragment that must be inserted in the API call of the calling module to get the address of the variable in memory, which holds the channel group information. Example value is `&Dio_ChannelGroupCfg[0]`. Only array index can be changed.

  This value will be assigned to the following symbolic names.

  - The symbolic name derived from the *DioChannelGroup* container short name prefixed with "*DioConf_DioChannelGroup_*".
- `DioPortMask` is a mask, which defines the positions of the channel group. The data type depends on the port width. Since it is not possible to identify the position of ChannelGroup, `DioPortMask` cannot be set to 0.
- `DioPortOffset` is a position of the Channel group on the port, counted from the LSB. `DioPortOffset` will be calculated automatically during the generation process from the mask value. Therefore, `DioPortOffset` is not changeable.

5 Functional Description

5.1 Initialization

The DIO Driver does not provide a function for port configuration and initialization. This must be done with the PORT Driver [3] prior to using the DIO Driver.

5.2 Runtime Reconfiguration

The DIO Driver is not runtime configurable. Changing the port pin direction during runtime is done by the PORT Driver.

5.3 Channels, Ports and Channel Groups

A channel represents a single general-purpose digital input/output pin. The value of a single channel is of the type `Dio_LevelType` and can either be `STD_HIGH`, which corresponds to physical high, or `STD_LOW`, which corresponds to physical low.

A port represents several channels, which are grouped by hardware and are controlled by one hardware register. The corresponding data type for a value of a port is `Dio_PortLevelType` whose size depends on the width of the largest port. When reading a port with a smaller width, the MSBs are set to '0'. Writing to a port with a smaller width ignores the upper bits.

A channel group consists of several adjoining channels within a port. Its value is also of the type `Dio_PortLevelType`.

A channel group is specified by three parameters: the port it belongs to (the *DioChannelGroup* container is inside the *DioPort* container), a mask, and an offset. The offset is derived from the mask during the generation process and does not need to be configured by the user.

The value of a channel group is aligned to the LSB, that is, the group's read and write services do the shifting and masking (See **Figure 3**).

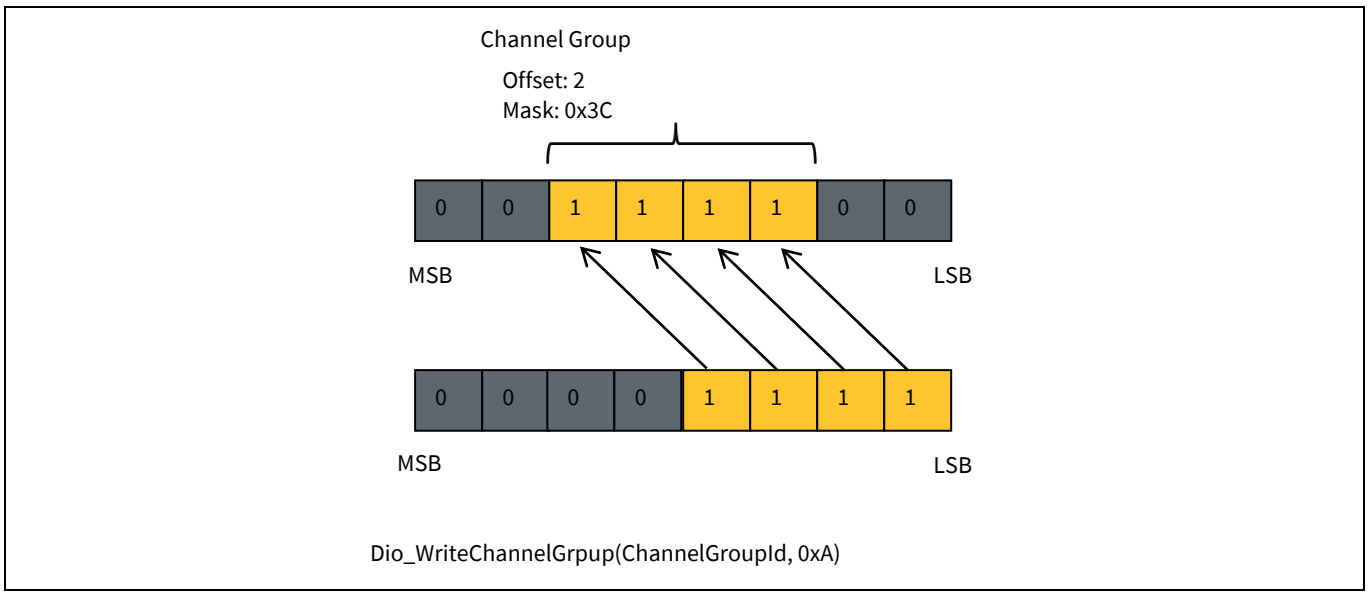

Figure 3 DIO Channel Groups Masking and Alignment

5.4 Write Services

The DIO Driver provides the following synchronous API functions to modify the level of output channels, ports, and groups:

- void `Dio_WriteChannel(Dio_ChannelType ChannelId, Dio_LevelType Level)`
- void `Dio_WritePort(Dio_PortType PortId, Dio_PortLevelType Level)`
- void `Dio_WriteChannelGroup(Dio_ChannelGroupType *ChannelGroupIdPtr, Dio_PortLevelType Level)`
- `Dio_LevelType Dio_FlipChannel(Dio_ChannelType ChannelId)`
- void `Dio_MaskedWritePort(Dio_PortType PortId, Dio_PortLevelType Level, Dio_PortLevelType Mask)`

All channels, ports, and channel groups are referred to by symbolic names, which must be configured. The following example demonstrates how to use the write services.

Code Listing 2 Code 1. Example Write Services

```
#include "Dio.h"
/* ... */
/* initialize ports */
/* .... */
Dio_LevelType c;
Dio_WriteChannel(DioConf_DioChannel_MY_LED_0, STD_HIGH);
Dio_WritePort(DioConf_DioPort_MY_LED_PORT, 0xFF);
Dio_WriteChannelGroup(DioConf_DioChannelGroup_MY_LED_GROUP_0, 0x0);
c = Dio_FlipChannel(DioConf_DioChannel_MY_LED_0);
Dio_MaskedWritePort(DioConf_DioPort_MY_LED_PORT, 0xFF, 0xAA);
```

- Moreover, writing to a channel group does not affect the remaining channels of the port, which are not members of the channel group.
- When writing to a port with a smaller width than the size of `Dio_PortLevelType`, the MSBs are ignored.

Note: The Port Pin output is not affected from writing value when port pin mode is other than GPIO.

Note: The Port Pin output is not affected from writing value during port pin direction is other than output. The written value will be reflected to the output level when port pin direction is changed to output.

5.5 Read Services

For reading channels, ports and channel groups, the driver offers the following synchronous functions:

- `Dio_LevelType Dio_ReadChannel(Dio_ChannelType ChannelId)`
- `Dio_PortLevelType Dio_ReadPort(Dio_PortType PortId)`
- `Dio_PortLevelType Dio_ReadChannelGroup(Dio_ChannelGroupType *ChannelGroupIdPtr)`

Channels, ports, and channel groups are also referred to by the symbolic names configured in the EB tresos Studio. The following example shows how to use the read services:

Code Listing 3 Code 2. Example Read Services

```
#include "Dio.h"
/* ... */
/* initialize ports */
/* .... */
Dio_LevelType c;
Dio_PortLevelType p;
Dio_PortLevelType g;
c = Dio_ReadChannel(DioConf_DioChannel_MY_DIP_SWITCH_0);
p = Dio_ReadPort(DioConf_DioPort_MY_DIP_PORT);
g = Dio_ReadChannelGroup(DioConf_DioChannelGroup_MY_DIP_SWITCH_GROUP_0);
```

- When reading from a port with a smaller width than the size of `Dio_PortLevelType`, the MSBs are set to 0.
- When reading ports with pins, which are not available in hardware, the missing pin levels are always returned as 0.
- When reading pins whose directions are input, actual input levels can be acquired.
- When reading pins whose directions are output and whose input buffers are enabled, actual output levels can be acquired.
- When reading pins whose directions are output and whose input buffers are disabled, output level settings can be acquired.
- When reading pins whose directions are input/output disabled, output level settings can be acquired.

5.6 API Parameter Checking

The driver's services perform regular error checks.

When an error occurs, the error hook routine (configured via `DioErrorCalloutFunction`) is called and the error code, as well as service ID, module ID, and instance ID are passed as parameters.

If default error detection is enabled, all errors are also reported to the DET, a central error hook function within the AUTOSAR environment. The checking itself cannot be deactivated for safety reasons.

The following development error checks are performed by the services of the DIO Driver:

- The `Dio_GetVersionInfo` function checks whether the `VersionInfo` pointer is NULL. In this case, the error code `DIO_E_PARAM_POINTER` is reported.
- The `Dio_ReadChannel`, `Dio_WriteChannel`, and `Dio_FlipChannel` functions check whether the `ChannelId` parameter is valid. In case of an invalid `ChannelId`, the error code `DIO_E_PARAM_INVALID_CHANNEL_ID` is reported.

Note: `DIO_E_PARAM_INVALID_CHANNEL_ID` will also be reported when the accessed port pin is not available on the derivative/target.

- `Dio_WriteChannel` additionally reports `DIO_E_PARAM_INVALID_LEVEL_VALUE` if the given level does not match `STD_HIGH` or `STD_LOW`.
- `Dio_ReadPort`, `Dio_WritePort`, `Dio_MaskedWritePort`, `Dio_ReadChannelGroup`, and `Dio_WriteChannelGroup` report the error code `DIO_E_PARAM_INVALID_PORT_ID` in case of an invalid `PortId`.

- Finally, for `Dio_ReadChannelGroup` and `Dio_WriteChannelGroup`, passing an invalid `ChannelGroupIdPtr` leads to the error code `DIO_E_PARAM_INVALID_GROUP`.

Furthermore, in case of an error all read services return 0 and all write services do not process the write request. This process is the same even if default error detection is disabled.

A complete list of all error codes is given in **7.1.3 Constants**.

5.7 Configuration Checking

The following conditions are checked when configuring the DIO Driver with EB tresos Studio:

- Channel group mask must fit into the port.

  The channel group mask describes a consecutive number of pins in the port, which make up the channel group. The lower limit for the mask is 1 and the upper limit is 2^(width of the port) - 1.

- Unique user names for DioPort, DioChannels, and DioChannelGroups.

  The DIO Driver generates macros of user-configured names for architecture-independent implementation. Ensure that no name is used more than once.

- Architecture specific port and port pin check.

  Check that the selected `DioPortId` and the configured channels in this port (bit number in this port) are defined for the hardware derivative selected.

5.8 Reentrancy

All services are reentrant, that is, they can be accessed by different upper layers or drivers concurrently.

5.9 Debugging Support

The DIO Driver does not support debugging.

6 Hardware Resources

6.1 Ports and Pins

The DIO Driver can be used all general-purpose pins of the Traveo II microcontroller as digital input/output.

Any port can operate in 8 pins or 1 pin units and all registers can be read or written in 1 bit or 32 bits units. For more details, refer to each Hardware Manual (See **Hardware Documentation** for the Hardware Manual version and subderivative).

6.2 Timer

The DIO Driver does not use any hardware timers.

6.3 Interrupts

The DIO Driver does not use any interrupts.

7 Appendix

7.1 API Reference

7.1.1 Include Files

The file *Dio.h* includes all necessary external identifiers. Therefore, your application only needs to include *Dio.h* to make all API functions and data types available.

7.1.2 Data Types

7.1.2.1 Dio_ChannelType

Type

`uint16`

Description

Type for storing channel IDs. For parameter values of type `Dio_ChannelType`, the symbolic names provided by the configuration description have to be used.

7.1.2.2 Dio_PortType

Type

`uint8`

Description

Type for storing port IDs. For parameter values of type `Dio_PortType`, the symbolic names provided by the configuration description have to be used.

7.1.2.3 Dio_ChannelGroupType

Type

`typedef struct`

Description

This type defines a struct describing a channel group. A channel group consists of several adjoining channels of the port. The mask contains the bit mask defining all channels of the port which are valid for the channel group. The mask is aligned with the LSB of the port. The offset gives the position of the first set bit in the bit mask, counted from the LSB of the port. See **Figure 3** for an example. For parameter values of type `Dio_ChannelGroupType`, the symbolic names provided by the configuration description have to be used.

7.1.2.4 Dio_LevelType

Type

`uint8`

Description

Type for representing the level of a channel.

7.1.2.5 Dio_PortLevelType

Type

`uint8`

Description

Type for storing values of a port.

7.1.2.6 Dio_ConfigType

Type

`typedef struct`

Description

`Dio_ConfigType` defines a structure which holds the DIO Driver configuration set.

7.1.3 Constants

7.1.3.1 Error Codes

The service might return the error codes, show in **Table 2**, if default error detection is enabled:

Table 2 Error Codes

Name	Value	Description
`DIO_E_PARAM_INVALID_CHANNEL_ID`	0x0A	Invalid `ChannelId` passed
`DIO_E_PARAM_INVALID_LEVEL_VALUE`	0x0B	Level value is not `STD_HIGH` or `STD_LOW`
`DIO_E_PARAM_CONFIG`	0x10	Invalid `ConfigPtr` passed
`DIO_E_PARAM_INVALID_PORT_ID`	0x14	Invalid `PortId` passed
`DIO_E_PARAM_INVALID_GROUP`	0x1F	Invalid `ChannelGroupIdPtr` passed
`DIO_E_PARAM_POINTER`	0x20	Invalid `VersionInfo` passed

7.1.3.2 Version Information

Table 3 lists the version information published in the Driver's header file.

Table 3 Version Information

Name	Value	Description
`DIO_SW_MAJOR_VERSION`	refer to release notes	Software Major version number
`DIO_SW_MINOR_VERSION`	refer to release notes	Software Minor version number
`DIO_SW_PATCH_VERSION`	refer to release notes	Software Patch version number

7.1.3.3 Module Information

Table 4 Module Information

Name	Value	Description
DIO_MODULE_ID	120	Module ID
DIO_VENDOR_ID	66	Vendor ID

7.1.3.4 API Service IDs

The API Service IDs, listed in **Table 5**, are published in the Driver's header file.

Table 5 API Service IDs

Name	Value	Description
DIO_API_READ_CHANNEL	0x00	Channel Read Service
DIO_API_WRITE_CHANNEL	0x01	Channel Write Service
DIO_API_READ_PORT	0x02	Port Read Service
DIO_API_WRITE_PORT	0x03	Port Write Service
DIO_API_READ_CHANNEL_GROUP	0x04	Channel Group Read Service
DIO_API_WRITE_CHANNEL_GROUP	0x05	Channel Group Write Service
DIO_API_FLIP_CHANNEL	0x11	Channel Flip Service
DIO_API_GET_VERSION_INFO	0x12	Version Read Service
DIO_API_MASKED_WRITE_PORT	0x20	Port Masked Write Service

7.1.4 Functions

7.1.4.1 Dio_ReadChannel

Syntax

```
Dio_LevelType Dio_ReadChannel
(
    Dio_ChannelType ChannelId
)
```

Service ID

0x00

Parameters (in)

- `ChannelId` - ID of DIO Channel.

Parameters (out)

None

Return value

- `STD_HIGH` - The physical level of the pin is high.
- `STD_LOW` - The physical level of the pin is low.

DET Errors

- `DIO_E_PARAM_INVALID_CHANNEL_ID` - `ChannelId` is invalid.

DEM Errors

None

Description

This function reads and returns the value of the channel specified by the parameter `ChannelId`.

7.1.4.2 Dio_WriteChannel

Syntax

```
void Dio_WriteChannel
(
   Dio_ChannelType ChannelId,
   Dio_LevelType Level
)
```

Service ID

0x01

Parameters (in)

- `ChannelId` - ID of DIO Channel.
- `Level`- Value to be written.

Parameters (out)

None

Return value

None

DET Errors

- `DIO_E_PARAM_INVALID_CHANNEL_ID` - `ChannelId` is invalid.
- `DIO_E_PARAM_INVALID_LEVEL_VALUE` - The Level value does not match `STD_LOW` or `STD_HIGH`.

DEM Errors

None

Description

This function writes the value Level to the channel specified by the parameter `ChannelId`.

7.1.4.3 Dio_ReadPort

Syntax

```
Dio_PortLevelType Dio_ReadPort
(
   Dio_PortType PortId
)
```

Service ID

0x02

Parameters (in)

- `PortId` - ID of DIO Port.

Parameters (out)

None

Return value

The value of the port.

DET Errors

- `DIO_E_PARAM_INVALID_PORT_ID` - `PortId` is invalid.

DEM Errors

None

Description

This function reads and returns the value of the port specified by the parameter `PortId`.

7.1.4.4 Dio_WritePort

Syntax

```
void Dio_WritePort
(
   Dio_PortType PortId,
   Dio_PortLevelType Level
)
```

Service ID

0x03

Parameters (in)

- `PortId` - ID of DIO Port.
- `Level` - Value to be written

Parameters (out)

None

Return value

None

DET Errors

- `DIO_E_PARAM_INVALID_PORT_ID` - The `PortId` is invalid.

DEM Errors

None

Description

This function writes the value Level to the port specified by the parameter `PortId`.

7.1.4.5 Dio_ReadChannelGroup

Syntax

```
Dio_PortLevelType Dio_ReadChannelGroup
(
   const Dio_ChannelGroupType * ChannelGroupIdPtr
)
```

Service ID

0x04

Parameters (in)

- `ChannelGroupIdPtr` - Pointer to DIO Channel Group.

Parameters (out)

None

Return value

The value of the channel group.

DET Errors

- `DIO_E_PARAM_INVALID_GROUP` - The `ChannelGroupIdPtr` is invalid.
- `DIO_E_PARAM_INVALID_PORT_ID` - The port referenced in the channel group is not configured.

DEM Errors

None

Description

This function reads and returns the value of a channel group specified by the parameter `ChannelGroupIdPtr`.

7.1.4.6 Dio_WriteChannelGroup

Syntax

```
void Dio_WriteChannelGroup
(
   const Dio_ChannelGroupType *ChannelGroupIdPtr,
   Dio_PortLevelType Level
)
```

Service ID

0x05

Parameters (in)

- `ChannelGroupIdPtr` - Pointer to DIO Channel group.
- `Level` - Value to be written.

Parameters (out)

None

Return value

None

DET Errors

- `DIO_E_PARAM_INVALID_GROUP` - The `ChannelGroupIdPtr` is invalid.
- `DIO_E_PARAM_INVALID_PORT_ID` - The port referenced in the channel group is not configured.

DEM Errors

None

Description

This function writes the value Level to the channel group specified by the parameter `ChannelGroupIdPtr`.

7.1.4.7 Dio_GetVersionInfo

Syntax

```
void Dio_GetVersionInfo
(
   Std_VersionInfoType* VersionInfo
)
```

Service ID

0x12

Parameters (in)

None

Parameters (out)

- `VersionInfo` - Pointer to where to store the version information of this module.

Return value

None

DET Errors

- `DIO_E_PARAM_POINTER` - The `VersionInfo` is NULL pointer.

DEM Errors

None

Description

This service returns module version, vendor and module ID information of the DIO Driver.

7.1.4.8 Dio_FlipChannel

Syntax

```
Dio_LevelType Dio_FlipChannel
(
   Dio_ChannelType ChannelId
)
```

Service ID

0x11

Parameters (in)

- `ChannelId` - ID of DIO Channel.

Parameters (out)

None

Return value

- `STD_HIGH` - The physical level of the pin is high.
- `STD_LOW` - The physical level of the pin is low.

DET Errors

- `DIO_E_PARAM_INVALID_CHANNEL_ID` - The `ChannelId` is invalid.

DEM Errors

None

Description

This function flips (change from 1 to 0 or from 0 to 1) the level of the specified channel by the parameter `ChannelId` and returns the level of the channel after flip.

7.1.4.9 Dio_MaskedWritePort

Syntax

```
void Dio_MaskedWritePort
(
   Dio_PortType PortId,
   Dio_PortLevelType Level,
   Dio_PortLevelType Mask
)
```

Service ID

0x20

Parameters (in)

- `PortId` - ID of DIO Port.
- `Level` - Value to be written.
- `Mask` - Bit mask to select the pins that shall be modified.

Parameters (out)

None

Return value

None

DET Errors

- `DIO_E_PARAM_INVALID_PORT_ID` - The `PortId` is invalid.

DEM Errors

None

Description

This service sets a subset of masked bits to a specified level.

7.1.5 Required Callback Functions

7.1.5.1 DET

If default error detection is enabled, the DIO Driver uses the following callback function that is provided by DET. If you do not use DET, you have to implement this function within your application.

7.1.5.1.1 Det_ReportError

Syntax

```
Std_ReturnType Det_ReportError
(
    uint16 ModuleId,
    uint8 InstanceId,
    uint8 ApiId,
    uint8 ErrorId
)
```

Reentrancy

Reentrant

Parameters (in)

- `ModuleId` - Module ID of calling module.
- `InstanceId` - Instance ID of calling module.
- `ApiId` - ID of the API service that calls this function.
- `ErrorId` - ID of the detected development error.

Return value

Returns always E_OK (is required for services).

Description

Service for reporting development errors.

7.1.5.2 Callout functions

7.1.5.2.1 Error callout API

The AUTOSAR DIO module requires an error callout handler. Each error is reported to this handler, error checking cannot be switched off. The name of the function to be called can be configured by parameter `DioErrorCalloutFunction`.

Syntax

```
void Error_Handler_Name
(
    uint16 ModuleId,
    uint8 InstanceId,
    uint8 ApiId,
    uint8 ErrorId
)
```

Reentrancy

Reentrant

Parameters (in)

- `ModuleId` - Module ID of calling module.
- `InstanceId` - Instance ID of calling module.
- `ApiId` - ID of the API service that calls this function.
- `ErrorId` - ID of the detected error.

Return value

None

Description

Service for reporting errors.

8 Appendix

8.1 Access Register Table

8.1.1 GPIO

Table 6 GPIO Access Register Table

Register	Bit No.	Access Size	Value	Description	Timing	Mask Value	Monitoring Value
GPIO_PRT_OUT (Port output data register)	31:0	Word (32 bits)	Depends on configuration value or API.	The output data for the IO pins in the port.	`Dio_ReadChannel` `Dio_ReadPort` `Dio_WritePort` `Dio_MaskedWritePort` `Dio_ReadChannelGroup` `Dio_WriteChannelGroup` `Dio_FlipChannel`	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
GPIO_PRT_OUT_CLR (Port output data clear register)	31:0	Word (32 bits)	Depends on configuration value or API	Clear output data of specific IO pins in the corresponding port to '0'.	`Dio_WriteChannel`	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
GPIO_PRT_OUT_SET (Port output data set register)	31:0	Word (32 bits)	Depends on configuration value or API	Set output data of specific IO pins in the corresponding port to '1'.	`Dio_WriteChannel`	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
GPIO_PRT_OUT_INV (Port output data invert register)	31:0	Word (32 bits)	Depends on configuration value or API	Invert output data of specific IO pins in the corresponding port.	`Dio_FlipChannel`	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
GPIO_PRT_IN (Port input state register)	31:0	Word (32 bits)	Depends on current pin status	Read current pin status for IO pins.	`Dio_ReadChannel` `Dio_ReadPort` `Dio_ReadChannelGroup` `Dio_FlipChannel`	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)
GPIO_PRT_CFG (Port configuration register)	31:0	Word (32 bits)	Depends on configuration value or API	Read pin direction for the IO pins.	`Dio_ReadChannel` `Dio_ReadPort` `Dio_ReadChannelGroup`	0x00000000 (Monitoring is not needed.)	0x00000000 (Monitoring is not needed.)

Revision history

Revision	Issue Date	Description of Change
**	05/17/2018	New Spec.
*A	09/27/2018	Added two Traveo II Automotive Body Controller High Family TRMs in Hardware Documentation. Deleted the data sheet in Hardware Documentation.
*B	12/17/2018	Update section 4.4 Channel Group Configuration. Revised description about channel group configuration.
*C	06/04/2019	Updated hardware documentation information. Updated link number for Layered Software Archtecture.
*D	09/04/2020	Changed a memmap file include folder in section "Memory Mapping".
*E	2020-11-20	MOVED TO INFINEON TEMPLATE.

Trademarks
All referenced product or service names and trademarks are the property of their respective owners.

Edition <2020-11>
Published by

Infineon Technologies AG

81726 Munich, Germany

© 2018-2020 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Go to www.cypress.com/support

Document reference
002-23342 Rev. *E

IMPORTANT NOTICE
The information given in this document shall in no event be regarded as a guarantee of conditions or characteristics (“Beschaffenheitsgarantie”) .

With respect to any examples, hints or any typical values stated herein and/or any information regarding the application of the product, Infineon Technologies hereby disclaims any and all warranties and liabilities of any kind, including without limitation warranties of non-infringement of intellectual property rights of any third party.

In addition, any information given in this document is subject to customer’s compliance with its obligations stated in this document and any applicable legal requirements, norms and standards concerning customer’s products and any use of the product of Infineon Technologies in customer’s applications.

The data contained in this document is exclusively intended for technically trained staff. It is the responsibility of customer’s technical departments to evaluate the suitability of the product for the intended application and the completeness of the product information given in this document with respect to such application.

For further information on the product, technology, delivery terms and conditions and prices please contact your nearest Infineon Technologies office (**www.infineon.com**).

WARNINGS
Due to technical requirements products may contain dangerous substances. For information on the types in question please contact your nearest Infineon Technologies office.

Except as otherwise explicitly approved by Infineon Technologies in a written document signed by authorized representatives of Infineon Technologies, Infineon Technologies’ products may not be used in any applications where a failure of the product or any consequences of the use thereof can reasonably be expected to result in personal injury.